

RepoSplit 2.0: Autonomous Enterprise Monolith Modernization Engine

Built for the IBM Bob 2.0 Hackathon · lablab.ai
Document Version: 2.0 (Master Unified Specification) | Date: August 2026

1. Executive Summary

RepoSplit 2.0 is an autonomous, AI-driven code transformation engine that converts monolithic legacy codebases into independently deployable, production-ready microservice architectures using IBM Bob 2.0’s full-repository context reasoning. While traditional microservice migrations require engineering teams months of manual dependency mapping, database refactoring, and risk analysis, RepoSplit 2.0 decomposes, partitions, tests, and deploys entire repositories in minutes.

Where prior hackathon tools focus purely on observing a codebase—visualizing structure or simulating risk—RepoSplit 2.0 acts directly on it, producing working service code, isolated database schemas, and infrastructure contracts as its primary output. RepoSplit 2.0 solves the hardest engineering bottlenecks of software refactoring: relational database state decoupling, distributed transaction scaffolding, live zero-regression traffic verification, and automated enterprise deployment.

2. Competitive Positioning vs. Prior Hackathon Winners

A review of winning submissions from prior IBM Bob hackathons reveals a consistent pattern: top-ranking projects excel at **observing** a codebase—analyzing provenance, mapping structure, or predicting deployment failures—rather than **acting** on it. RepoSplit 2.0 is deliberately positioned to fill this open competitive gap by serving as an active transformation engine.

Prior Winning Project	Category	Operational Function	Core Limitation	RepoSplit 2.0 Paradigm Shift
Pedigree (1st Place)	Provenance	Cryptographical ly signs AI commits and	Observes metadata; does not modify	Acts directly on source code to refactor

Prior Winning Project	Category	Operational Function	Core Limitation	RepoSplit 2.0 Paradigm Shift
		verifies author models.	source code.	monoliths into microservices.
Atlas (2nd Place)	Visualization	Renders GitHub repositories as interactive 3D city maps.	Provides visual insight; does not restructure architecture.	Generates concrete microservice code , DB schemas, and API routes.
Sandbox (3rd Place)	Risk Simulation	Predicts and replays cascading deployment failures before release.	Identifies risk; requires manual engineering fixes.	Auto-heals code mismatches via eBPF shadow traffic validation loops.

3. Problem Statement & Target Audience

E-commerce and enterprise monoliths hit a predictable breaking point as they scale: a single traffic spike—such as a flash sale or holiday rush—can take down checkout, inventory, and billing simultaneously because they all run as one tightly coupled process.

Splitting a monolith into microservices is one of the most requested, and most avoided, engineering refactors for three structural reasons:

- **Tangled Code & Hidden Dependencies:** Undocumented coupling across hundreds of legacy files makes manual dependency mapping labor-intensive and prone to error.
- **Shared Database State:** Monoliths rely on single SQL databases with inter-table foreign keys, rendering simple code separation insufficient without schema partitioning.
- **Production Downtime & Regression Risk:** Engineers fear introducing breaking changes or missing subtle edge-case logic during service extraction.

Target User: Engineering teams at small-to-mid-sized e-commerce companies (roughly 5–50 engineers) whose codebase has outgrown its original monolithic architecture.

4. Pipeline Workflow & Multi-Agent Architecture

4.1 The 5-Stage Transformation Pipeline

1. **Ingest:** IBM Bob 2.0 reads the full repository—routes, models, database schemas, and business logic.
2. **Map:** Builds a complete dependency graph showing function call paths across inventory, pricing, checkout, and orders.
3. **Isolate:** Flags tightly coupled logic, partitions shared database schemas, and proposes clean service boundaries.
4. **Scaffold:** Generates independent service code, data access layers, gRPC/REST bindings, and Saga event handlers.
5. **Explain:** Produces a human-readable migration report describing what moved, what depends on what, and what is risky.

4.2 Parallel Multi-Agent Subsystems

To execute this pipeline, RepoSplit 2.0 spawns specialized subagents running in parallel:

Subagent	Role & Responsibility	Primary Artifact Produced
Architect Agent	Reads the entire repository AST, maps module call graphs, identifies domain boundaries, and produces a risk-scored split plan.	Live dependency graph & migration plan.
Data Agent (DataSplit Engine)	Analyzes ORM schemas, SQL queries, and foreign key relations to partition shared databases into domain-isolated schemas and emit Saga event patterns.	Isolated DB schemas, migration scripts, Saga pattern events.
Contract Agent	Defines inter-service interfaces, generating OpenAPI 3.0 specifications, gRPC .proto definitions, and AsyncAPI specs.	OpenAPI 3.0 docs, gRPC Proto definitions, AsyncAPI specs.
Test & Refactor Agent	Generates parity integration test suites and patches code	Integration test suites &

Subagent	Role & Responsibility	Primary Artifact Produced
	syntax or payload mismatches automatically.	auto-healing code patches.

5. Comprehensive Feature Set

- **Autonomous Code Transformation Engine (Core):** Takes a monolithic repository and produces deployable FastAPI/Go microservice code complete with API routes, data access layers, and auto-generated Swagger documentation.
- **DataSplit Engine (Database & State Decoupling):** Extracts entity clusters into domain-isolated schemas and scaffolds asynchronous Outbox/Saga messaging patterns via NATS/Kafka for cross-domain operations.
- **Zero-Regression Shadow Traffic Parity:** Sidecar proxies (eBPF/Envoy) fork live incoming monolith requests to both the legacy monolith and new microservices in parallel. If response payloads differ, Bob 2.0 analyzes the execution path, patches the missing business logic, and re-validates automatically.
- **Live Dependency Graph Visualization:** An interactive Streamlit/D3.js graph renders monolith modules as connected nodes that visibly separate into color-coded service clusters with severed calls highlighted.
- **Automated Risk Scoring:** Quantifies migration confidence based on severed cross-cluster calls: Low Risk (0–1 calls cut), Medium Risk (2–4 calls cut), and High Risk (5+ calls cut).
- **Natural-Language Codebase Q&A:** Allows developers to ask Bob 2.0 questions about its architectural choices (e.g., "Why did you separate pricing from inventory?") and receive responses grounded in actual code context.
- **Load Simulation Preview:** A comparison chart illustrates the business impact by displaying the monolith's estimated failure point versus the throughput capacity of the newly split services under load.

6. Technical Stack & Architecture Layers

Layer	Component	Technology Applied
Input	Legacy Monolith Repository	Sample Flask / Django

Layer	Component	Technology Applied
		E-commerce codebase
AI Analysis	Multi-Agent Reasoning Engine	IBM Bob 2.0 API & Bob Shell CLI
Generated Services	Isolated Microservices	Python (FastAPI) / Go
State & Messaging	Partitioned DBs & Event Streaming	PostgreSQL / Redis / NATS Streaming
Orchestration	Cluster Deployment	Docker Compose, Red Hat OpenShift Helm Charts
Observability	Distributed Tracing	Instana & OpenTelemetry (traceparent header injection)
Presentation	Visual Dashboard & Interface	Streamlit + Plotly / D3.js Graph Visualization

7. Feature Evolution: Version 1.0 vs. Version 2.0

Feature Area	RepoSplit 1.0 (Base Concept)	RepoSplit 2.0 (Master Specification)
Code Splitting	Scaffolds isolated service endpoints.	Scaffolds endpoints + injects RPC client communication layers.
Data Layer	Shared monolith database assumption.	Autonomous database schema partitioning & Saga pattern injection.

Feature Area	RepoSplit 1.0 (Base Concept)	RepoSplit 2.0 (Master Specification)
Agent Architecture	Single-prompt repository analysis.	Parallel multi-agent workflow (Architect, Data, Contract, Test).
Validation Method	Simulated load performance chart.	Real-time eBPF Shadow Traffic proxy with self-healing feedback loop.
Deployment Assets	Basic Docker Compose files.	Red Hat OpenShift Helm Charts & Instana OpenTelemetry integration.
Developer Interface	Web UI summary.	Web UI + Bob Shell CLI + Automated GitHub Multi-Repo Pull Requests.

8. Delivering the 2032 Industry Standard Today

Systems that understand full codebases, make independent architectural risk calls, partition relational databases, heal output regressions, and anticipate business outcomes are widely assumed to require years of industry progress. RepoSplit 2.0 proves that with IBM Bob 2.0's full-repository reasoning engine, this capability is operational today.

Capability Assumed to Require Years of Progress	Delivered in RepoSplit 2.0
Autonomous Architectural Restructuring	Working microservice code and isolated DB schemas generated directly by Bob 2.0.
Whole-System AST Comprehension	Live dependency graph generated directly from Bob 2.0 whole-repo analysis.
Production Risk Judgment	Automated Low/Medium/High risk scoring based on severed cross-cluster interactions.

Capability Assumed to Require Years of Progress	Delivered in RepoSplit 2.0
Transparent Reasoning	Natural-language Q&A assistant explaining architectural choices on demand.
Forward-Looking Business Impact	Load simulation chart comparing monolith failure points with microservice throughput.

9. Business Value & Revenue Model

- **Addressable Market:** Millions of active online stores and enterprise systems worldwide remain built as single, tangled codebases.
- **Quantifiable ROI:** Manual microservice migrations cost \$250,000+ and take 6–12 months. RepoSplit 2.0 cuts initial dependency mapping, schema partitioning, and code scaffolding from months to under 10 minutes.
- **Monetization Model:** Per-repository fee for one-time analysis and split, plus an optional ongoing SaaS subscription for continuous architecture monitoring and drift detection.

10. Alignment with lablab.ai Judging Criteria

Judging Criterion	How RepoSplit 2.0 Delivers
Presentation	Clear problem statement, live 3D dependency graph untangling on stage, and a visible before/after code transformation moment.
Business Value	Defined target user, broad addressable market, concrete revenue model, and quantified savings in engineering hours.
Application of Technology	Deep usage of Bob 2.0's full-repository reasoning, Bob Shell CLI automation, and parallel multi-agent execution.

Judging Criterion	How RepoSplit 2.0 Delivers
Originality	Fills the competitive gap left by prior winners (Pedigree, Atlas, Sandbox) by delivering an active transformation engine.